

PHP AJAX CRUD Project

MySQL • jQuery AJAX • Bootstrap • Image Upload & Preview • Gender & Hobbies •
State-City Dependent Dropdown

A fully AJAX-driven build: Add, Edit, Delete and the users table all happen without a page reload, using plain procedural PHP endpoints under an ajax/ folder and jQuery on the front end.

1. Project Structure

Every AJAX endpoint lives in its own file under the ajax/ folder, keeping the front-end page (index.php) focused purely on markup.

```
php-crud/
|
|-- index.php
|-- db.php
|
|-- ajax/
|   |-- save.php
|   |-- list.php
|   |-- get.php
|   |-- update.php
|   |-- delete.php
|   |-- cities.php
|
|-- uploads/
|
|-- js/
|   |-- script.js
```

2. Database Setup

2.1 Create the Database

```
CREATE DATABASE php_crud;

USE php_crud;
```

2.2 Users Table

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  image VARCHAR(255),
  gender VARCHAR(20),
  hobbies VARCHAR(255),
  state_id INT,
  city_id INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.3 States and Cities Tables

For the State → City AJAX relationship, two supporting tables are needed:

```
CREATE TABLE states (
  id INT AUTO_INCREMENT PRIMARY KEY,
  state_name VARCHAR(100) NOT NULL
);

CREATE TABLE cities (
  id INT AUTO_INCREMENT PRIMARY KEY,
  city_name VARCHAR(100) NOT NULL,
  state_id INT NOT NULL
);
```

2.4 Sample States

```
INSERT INTO states (state_name) VALUES
('Delhi'),
('Maharashtra'),
('Uttar Pradesh'),
('Rajasthan');
```

2.5 Sample Cities

```
INSERT INTO cities (city_name, state_id) VALUES
('New Delhi', 1),
('North Delhi', 1),
('South Delhi', 1),

('Mumbai', 2),
('Pune', 2),
('Nagpur', 2),

('Lucknow', 3),
('Kanpur', 3),
('Noida', 3),

('Jaipur', 4),
('Jodhpur', 4),
('Udaipur', 4);
```

3. Database Connection — db.php

db.php

```
<?php

$conn = mysqli_connect("localhost", "root", "", "php_crud");

if (!$conn) {
    die("Database connection failed: " . mysqli_connect_error());
}

mysqli_set_charset($conn, "utf8mb4");

?>
```

4. Main Page — index.php

index.php

index.php only renders the form and an empty container for the users table. There is no server-rendered list here — the table is loaded entirely by AJAX after the page loads. The states dropdown is the one exception, rendered directly with PHP since it rarely changes.

```
<?php
include "db.php";

$states = mysqli_query($conn, "SELECT * FROM states ORDER BY state_name");
?>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

4.1 The Form (uses a hidden id field for Add vs. Edit)

The form has no action or method attribute — its submission is intercepted entirely by JavaScript. A hidden #user_id field is empty for Add and populated for Edit, which is how script.js decides whether

to call save.php or update.php.

```
<form id="userForm" enctype="multipart/form-data">

    <input type="hidden" name="id" id="user_id">

    <input type="text" name="name" id="name" class="form-control">
    <input type="email" name="email" id="email" class="form-control">

    <input type="file" name="image" id="image" class="form-control" accept="image/*">
    <img id="imagePreview" src="" width="80" style="display:none;">

    <input class="form-check-input" type="radio" name="gender" value="Male" id="male">
    <input class="form-check-input" type="radio" name="gender" value="Female" id="female">

    <input class="form-check-input" type="checkbox" name="hobbies[]" value="Reading" id="reading">
    <input class="form-check-input" type="checkbox" name="hobbies[]" value="Music" id="music">
    <input class="form-check-input" type="checkbox" name="hobbies[]" value="Sports" id="sports">
    <input class="form-check-input" type="checkbox" name="hobbies[]" value="Travel" id="travel">

    <select name="state_id" id="state_id" class="form-select">
        <option value="">Select State</option>
        <?php while ($state = mysqli_fetch_assoc($states)) { ?>
            <option value="<?php echo $state['id']; ?>">
                <?php echo htmlspecialchars($state['state_name']); ?>
            </option>
        <?php } ?>
    </select>

    <select name="city_id" id="city_id" class="form-select">
        <option value="">Select City</option>
    </select>

    <button type="submit" id="saveBtn" class="btn btn-primary">Save</button>
    <button type="button" id="cancelBtn" class="btn btn-secondary" style="display:none;">
        Cancel
    </button>
</form>
```

4.2 The Table Container

This empty div is filled by AJAX on page load and refreshed after every Add, Edit, or Delete.

```
<div id="message"></div>

<div id="userTable"></div>

<script src="js/script.js"></script>
```

5. Create — ajax/save.php

ajax/save.php

Handles the INSERT. Called by AJAX when the hidden id field is empty. Returns the plain string "success", or an error message, which script.js checks directly.

```
<?php

include "../db.php";

$name      = trim($_POST['name'] ?? '');
$email     = trim($_POST['email'] ?? '');
$gender    = $_POST['gender'] ?? '';
$state_id  = $_POST['state_id'] ?? '';
$city_id   = $_POST['city_id'] ?? '';

$hobbies = "";
if (isset($_POST['hobbies'])) {
    $hobbies = implode(",", $_POST['hobbies']);
}

if ($name == '') { echo "Name is required"; exit; }
if ($email == '') { echo "Email is required"; exit; }

$imageName = "";
if (isset($_FILES['image']) && $_FILES['image']['error'] == 0) {
    $imageName = time() . "_" . basename($_FILES['image']['name']);
    $uploadPath = "../uploads/" . $imageName;
    move_uploaded_file($_FILES['image']['tmp_name'], $uploadPath);
}

$name      = mysqli_real_escape_string($conn, $name);
$email     = mysqli_real_escape_string($conn, $email);
$gender    = mysqli_real_escape_string($conn, $gender);
$hobbies   = mysqli_real_escape_string($conn, $hobbies);

$sql = "INSERT INTO users
        (name, email, image, gender, hobbies, state_id, city_id)
        VALUES
        ('$name', '$email', '$imageName', '$gender', '$hobbies', '$state_id', '$city_id')";

if (mysqli_query($conn, $sql)) {
    echo "success";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>
```

6. Read — ajax/list.php

ajax/list.php

Returns the users table as an HTML fragment (not JSON). script.js drops the raw response straight into #userTable. This runs once on page load and again after every successful Add, Edit, or Delete.

```
<?php

include "../db.php";

$sql = "SELECT
        users.*,
        states.state_name,
        cities.city_name
```

```

FROM users
LEFT JOIN states ON users.state_id = states.id
LEFT JOIN cities ON users.city_id = cities.id
ORDER BY users.id DESC";

$result = mysqli_query($conn, $sql);

?>
<table class="table table-bordered table-striped">
  <thead>
    <tr>
      <th>ID</th><th>Image</th><th>Name</th><th>Email</th>
      <th>Gender</th><th>Hobbies</th><th>State</th><th>City</th><th>Action</th>
    </tr>
  </thead>
  <tbody>
    <?php if (mysqli_num_rows($result) > 0) { ?>
      <?php while ($row = mysqli_fetch_assoc($result)) { ?>
        <tr>
          <td><?php echo $row['id']; ?></td>
          <td>
            <?php if (!empty($row['image'])) { ?>
              
            <?php } else { ?>
              No Image
            <?php } ?>
          </td>
          <td><?php echo htmlspecialchars($row['name']); ?></td>
          <td><?php echo htmlspecialchars($row['email']); ?></td>
          <td><?php echo htmlspecialchars($row['gender']); ?></td>
          <td><?php echo htmlspecialchars($row['hobbies']); ?></td>
          <td><?php echo htmlspecialchars($row['state_name']); ?></td>
          <td><?php echo htmlspecialchars($row['city_name']); ?></td>
          <td>
            <button class="btn btn-sm btn-warning editBtn"
              data-id="<?php echo $row['id']; ?>">Edit</button>
            <button class="btn btn-sm btn-danger deleteBtn"
              data-id="<?php echo $row['id']; ?>">Delete</button>
          </td>
        </tr>
      <?php } ?>
    <?php } else { ?>
      <tr><td colspan="9" class="text-center">No users found</td></tr>
    <?php } ?>
  </tbody>
</table>

```

7. State → City AJAX — ajax/cities.php

ajax/cities.php

This is the core of the dependent dropdown. It's called both when the user manually changes the State field, and internally by the Edit flow to repopulate cities for the record being edited.

```

<?php

include "../db.php";

$state_id = $_POST['state_id'] ?? '';

if ($state_id == '') {
  echo '<option value="">Select City</option>';
  exit;
}

```

```

$state_id = (int)$state_id;

$sql = "SELECT * FROM cities WHERE state_id = $state_id ORDER BY city_name";
$result = mysqli_query($conn, $sql);

echo '<option value="">Select City</option>';

while ($row = mysqli_fetch_assoc($result)) {
    echo '<option value="' . $row['id'] . '">'
        . htmlspecialchars($row['city_name'])
        . '</option>';
}
?>

```

So when the user does:

```

State
|
Maharashtra
|
AJAX
|
cities.php
|
Mumbai
Pune
Nagpur

```

...the page doesn't reload.

8. Fetch a Single User — ajax/get.php

ajax/get.php

Called when the user clicks Edit. Unlike list.php and cities.php, this endpoint returns JSON, since script.js needs to read individual fields (name, email, gender, hobbies array, and so on) to populate the form.

```
<?php

include "../db.php";

$id = (int)($_POST['id'] ?? 0);

$sql = "SELECT * FROM users WHERE id = $id";
$result = mysqli_query($conn, $sql);

if ($row = mysqli_fetch_assoc($result)) {

    $row['hobbies'] = !empty($row['hobbies'])
        ? explode(",", $row['hobbies'])
        : [];

    echo json_encode($row);

} else {
    echo json_encode(["error" => "User not found"]);
}
?>
```

9. Update — ajax/update.php

ajax/update.php

Mirrors save.php, but targets an existing row by id and only replaces the image column when a new file was actually uploaded, deleting the old image file in that case.

```
<?php

include "../db.php";

$id = (int)($_POST['id'] ?? 0);

$name = trim($_POST['name'] ?? '');
$email = trim($_POST['email'] ?? '');
$gender = $_POST['gender'] ?? '';
$state_id = $_POST['state_id'] ?? '';
$city_id = $_POST['city_id'] ?? '';

$hobbies = "";
if (isset($_POST['hobbies'])) {
    $hobbies = implode(",", $_POST['hobbies']);
}

if ($id <= 0) { echo "Invalid ID"; exit; }
if ($name == '') { echo "Name is required"; exit; }
if ($email == '') { echo "Email is required"; exit; }

$name = mysqli_real_escape_string($conn, $name);
$email = mysqli_real_escape_string($conn, $email);
$gender = mysqli_real_escape_string($conn, $gender);
$hobbies = mysqli_real_escape_string($conn, $hobbies);

$oldImage = "";
```

```

$result = mysqli_query($conn, "SELECT image FROM users WHERE id = $id");
if ($row = mysqli_fetch_assoc($result)) {
    $oldImage = $row['image'];
}

$imageSQL = "";

if (isset($_FILES['image']) && $_FILES['image']['error'] == 0) {

    $imageName = time() . "_" . basename($_FILES['image']['name']);
    $uploadPath = "../uploads/" . $imageName;
    move_uploaded_file($_FILES['image']['tmp_name'], $uploadPath);

    if (!empty($oldImage) && file_exists("../uploads/" . $oldImage)) {
        unlink("../uploads/" . $oldImage);
    }

    $imageSQL = ", image='$imageName'";
}

$sql = "UPDATE users SET
    name='$name', email='$email', gender='$gender',
    hobbies='$hobbies', state_id='$state_id', city_id='$city_id'
    $imageSQL
    WHERE id=$id";

if (mysqli_query($conn, $sql)) {
    echo "success";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>

```

10. Delete — ajax/delete.php

ajax/delete.php

Deletes the database row and its associated image file, then reports back "success" so script.js can refresh the table.

```

<?php

include "../db.php";

$id = (int)($_POST['id'] ?? 0);

if ($id <= 0) { echo "Invalid ID"; exit; }

$result = mysqli_query($conn, "SELECT image FROM users WHERE id=$id");

$image = "";
if ($row = mysqli_fetch_assoc($result)) {
    $image = $row['image'];
}

$sql = "DELETE FROM users WHERE id=$id";

if (mysqli_query($conn, $sql)) {

    if (!empty($image) && file_exists("../uploads/" . $image)) {
        unlink("../uploads/" . $image);
    }

    echo "success";
}

```

```
} else {  
    echo "Error: " . mysqli_error($conn);  
}  
?>
```

11. Client-Side Logic — js/script.js

js/script.js

This is where nearly all of the interactivity lives: loading the table, the dependent city dropdown, submitting Add/Edit through one form, populating the form on Edit, deleting rows, and previewing the selected image before upload.

11.1 Load users on page open, and after every change

```
$(document).ready(function () {  
  
    loadUsers();  
  
    function loadUsers() {  
        $.ajax({  
            url: "ajax/list.php",  
            type: "GET",  
            success: function (response) {  
                $("#userTable").html(response);  
            }  
        });  
    }  
});
```

11.2 State → City dropdown

```
$("#state_id").change(function () {  
  
    let state_id = $(this).val();  
  
    $("#city_id").html('<option value="">Loading...</option>');  
  
    if (state_id == "") {  
        $("#city_id").html('<option value="">Select City</option>');  
        return;  
    }  
  
    $.ajax({  
        url: "ajax/cities.php",  
        type: "POST",  
        data: { state_id: state_id },  
        success: function (response) {  
            $("#city_id").html(response);  
        }  
    });  
});
```

11.3 Add / Update — one form, two endpoints

The hidden #user_id field decides which endpoint the form posts to: empty means save.php (Create), populated means update.php (Update).

```
$("#userForm").submit(function (e) {  
  
    e.preventDefault();  
  
    let formData = new FormData(this);  
    let id = $("#user_id").val();  
    let url = "ajax/save.php";  
  
    if (id != "") {  
        url = "ajax/update.php";  
    }  
});
```

```

$.ajax({
  url: url,
  type: "POST",
  data: formData,
  contentType: false,
  processData: false,
  success: function (response) {

    if (response.trim() == "success") {
      $("#message").html(
        '<div class="alert alert-success">Saved successfully</div>'
      );
      resetForm();
      loadUsers();
    } else {
      $("#message").html(
        '<div class="alert alert-danger">' + response + '</div>'
      );
    }
  },
  error: function () {
    $("#message").html(
      '<div class="alert alert-danger">Something went wrong</div>'
    );
  }
});
});

```

11.4 Edit — fetch JSON, then fill the form

After loading the user's data, it fires a second AJAX call to cities.php to rebuild the city list for that user's state, then selects the correct city once the list arrives.

```

$(document).on("click", ".editBtn", function () {

  let id = $(this).data("id");

  $.ajax({
    url: "ajax/get.php",
    type: "POST",
    dataType: "json",
    data: { id: id },
    success: function (data) {

      if (data.error) { alert(data.error); return; }

      $("#user_id").val(data.id);
      $("#name").val(data.name);
      $("#email").val(data.email);

      $("input[name='gender']").prop("checked", false);
      $("input[name='gender'][value='" + data.gender + "']").prop("checked", true);

      $("input[name='hobbies[]']").prop("checked", false);
      if (data.hobbies) {
        data.hobbies.forEach(function (hobby) {
          $("input[name='hobbies[]'][value='" + hobby + "']").prop("checked", true);
        });
      }

      $("#state_id").val(data.state_id);

      $.ajax({
        url: "ajax/cities.php",

```

```

        type: "POST",
        data: { state_id: data.state_id },
        success: function (response) {
            $("#city_id").html(response);
            $("#city_id").val(data.city_id);
        }
    });

    if (data.image != "") {
        $("#imagePreview").attr("src", "uploads/" + data.image).show();
    } else {
        $("#imagePreview").hide();
    }

    $("#formTitle").text("Edit User");
    $("#saveBtn").text("Update");
    $("#cancelBtn").show();

    $("html, body").animate(
        { scrollTop: $("#userForm").offset().top }, 500
    );
}
});
});

```

11.5 Delete

```

$(document).on("click", ".deleteBtn", function () {

    let id = $(this).data("id");

    if (!confirm("Are you sure you want to delete this user?")) {
        return;
    }

    $.ajax({
        url: "ajax/delete.php",
        type: "POST",
        data: { id: id },
        success: function (response) {

            if (response.trim() == "success") {
                $("#message").html(
                    '<div class="alert alert-success">Deleted successfully</div>'
                );
                loadUsers();
            } else {
                alert(response);
            }
        }
    });
});

```

11.6 Cancel, Reset Form, and Image Preview

```

$("#cancelBtn").click(function () {
    resetForm();
});

function resetForm() {
    $("#userForm")[0].reset();
    $("#user_id").val("");
    $("#city_id").html('<option value="">Select City</option>');
    $("#imagePreview").hide();
    $("#formTitle").text("Add User");
    $("#saveBtn").text("Save");
}

```

```
        $("#cancelBtn").hide();
    }

    $("#image").change(function () {
        let file = this.files[0];
        if (file) {
            let reader = new FileReader();
            reader.onload = function (e) {
                $("#imagePreview").attr("src", e.target.result).show();
            };
            reader.readAsDataURL(file);
        }
    });
});

});
```

12. How the Complete Flow Works

12.1 Add

```
Fill Form
|
| Submit
|
| JavaScript prevents page reload
|
| FormData()
|
| AJAX
|
| save.php
|
| MySQL INSERT
|
| loadUsers()
|
| Table refresh
```

12.2 State → City

This is the part you specifically asked for:

```
Select State
|
| $("#state_id").change()
|
| AJAX POST
|
| ajax/cities.php
|
| SELECT cities WHERE state_id = ?
|
| HTML <option>
|
| $("#city_id").html(response)
```

For example, selecting Maharashtra:

```
State
[ Maharashtra v ]

City
[ Mumbai      ]
[ Pune        ]
[ Nagpur       ]
```

Then changing to Uttar Pradesh:

```
State
[ Uttar Pradesh v ]

City
[ Lucknow     ]
[ Kanpur      ]
[ Noida       ]
```

No page refresh is required.

12.3 Edit

```
Click Edit
|
get.php
|
Get user JSON
|
Fill form
|
Select state
|
cities.php AJAX
|
Select user's city
|
Update
```

12.4 Delete

```
Click Delete
|
Confirmation
|
AJAX
|
delete.php
|
Delete DB record
|
Delete image
|
Refresh table
```

Note: This structure is intentionally simple/minimal. For a real application, the next two improvements would be: (1) use prepared statements instead of putting values directly into SQL, and (2) validate the image MIME type and size on the server rather than trusting the filename. For learning PHP procedural + AJAX CRUD, though, this structure gives a clean starting point and makes the State → City AJAX relationship easy to understand.